Theo Van Celst

Tema 1 Tarea Desarrollo Web

Desarrollo Web en Entorno Cliente

Theo Van Celst

ÍNDICE

Parte A: Arquitectura Cliente-Servidor **2**
1.1 Arquitectura de Dos Capas 2
1.2 Arquitectura de Tres Capas 2
1.3 Arquitectura de n capas 3
Parte B: Protocolos de Desarrollo Web **4**
1. Breve historia y evolución 4
2. Cómo funciona y para qué se utiliza 4
3. Ventajas y riesgos de seguridad 5
4. Ejemplos prácticos de uso documentados 5
5. Comparativa breve: HTTPS vs SSH 7
Uso de la Inteligencia Artificial 7
Bibliografía 8

Parte A: Arquitectura Cliente-Servidor

La división en capas en el desarrollo web permite separar las responsabilidades de una aplicación (interfaz, lógica y datos) para mejorar su organización, mantenibilidad y escalabilidad.

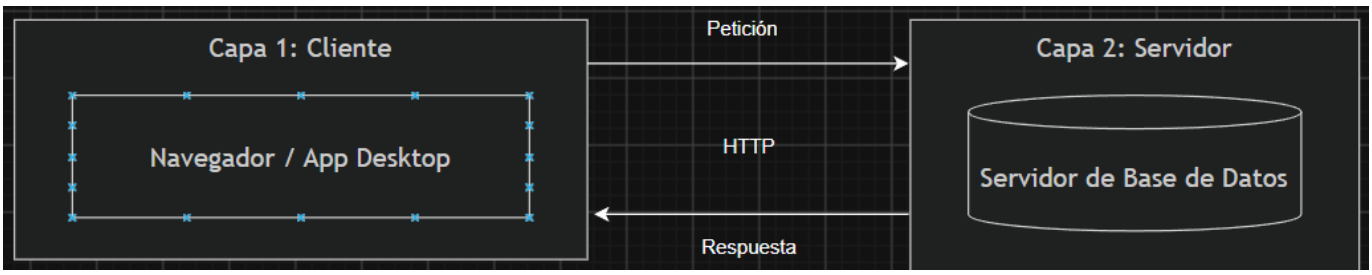
1.1 Arquitectura de Dos Capas

Descripción y flujo de información:

Es un modelo donde el **cliente** gestiona la interfaz de usuario y la lógica de negocio, comunicándose directamente con la **base de datos** (servidor de datos) a través de red local o protocolo dedicado.

Ventajas y desventajas:

- **Ventajas:** Desarrollo y despliegue muy simples, bajos costes de infraestructura y baja latencia inicial.
- **Desventajas:** Pobre escalabilidad, imposible de escalar horizontalmente y alto acoplamiento que dificulta el mantenimiento.



Un ejemplo real es la página web estática de un pequeño comercio local. Muestra horarios e información fija. No necesita guardar usuarios ni procesar pagos.

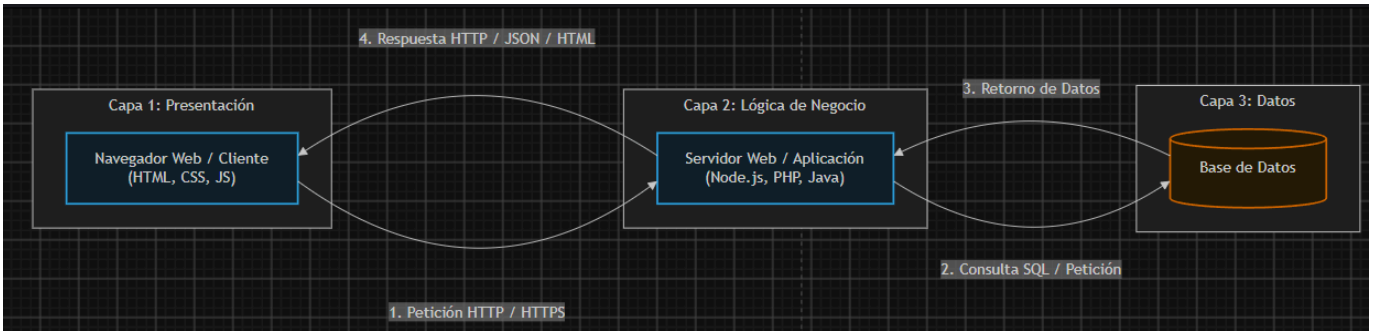
1.2 Arquitectura de Tres Capas

Descripción y flujo de información:

Este modelo divide la aplicación en tres partes independientes. La primera capa es la interfaz en el navegador. La segunda capa es el servidor web con la lógica de negocio. La tercera capa es la base de datos. El flujo empieza cuando el usuario envía un formulario. El servidor procesa la petición, consulta la base de datos y genera la vista final.

Ventajas y desventajas:

- **Ventajas:** Excelente separación de responsabilidades (Front/Back), mayor seguridad (BD aislada de la red pública) y mantenimiento/escalado sencillo por componentes.
- **Desventajas:** Configuración inicial y despliegue más complejos, y costes de infraestructura ligeramente superiores.



Un ejemplo claro es un e-commerce hecho con WooCommerce y WordPress. La vista se muestra en el navegador, PHP ejecuta la compra en el servidor y MySQL guarda los productos.

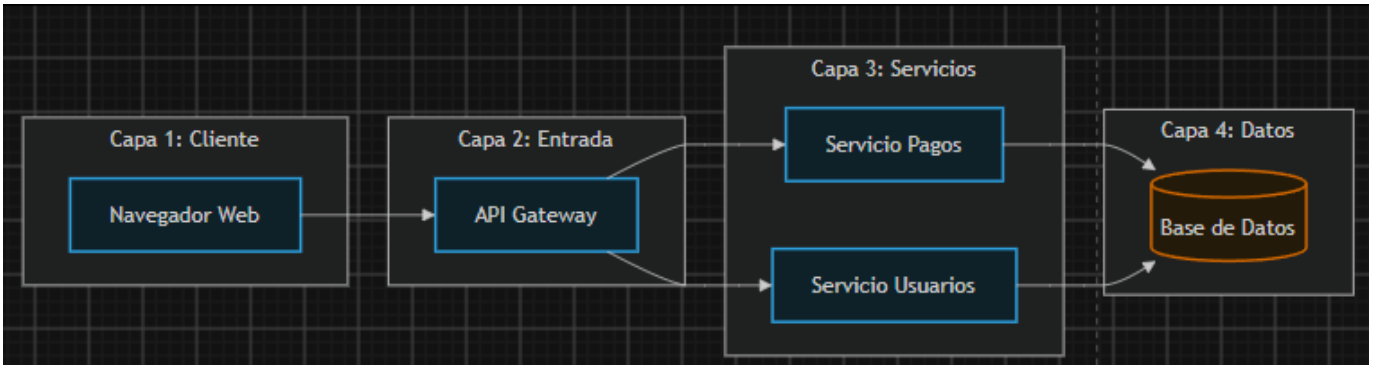
1.3 Arquitectura de n capas

Descripción y flujo de información:

Evolución de la arquitectura de tres capas que fragmenta la lógica y los datos en **múltiples servicios independientes** (balanceadores de carga, microservicios, capas de caché, API Gateways, bases de datos distribuidas) para gestionar sistemas de gran escala y alta disponibilidad.

Ventajas e Inconvenientes:

- **Ventajas:** Máxima escalabilidad horizontal, alta disponibilidad y tolerancia a fallos. Permite desplegar y actualizar módulos aislados sin detener el resto del sistema.
- **Inconvenientes:** Elevada complejidad técnica, dificultad de monitorización y depuración, e infraestructura y costes de mantenimiento elevados.



Un ejemplo real es la plataforma Netflix. Un servicio maneja los pagos, otro gestiona las recomendaciones y otro envía el vídeo.

Criterio	Arquitectura de 2 Capas	Arquitectura de 3 Capas	Arquitectura de N Capas
Escalabilidad	Muy reducida. Se colapsa con muchos usuarios.	Buena. Se escala el servidor y la base de datos por separado.	Excelente. Se escala cada microservicio según la necesidad.
Mantenimiento	Fácil al inicio, pero difícil si el código crece.	Organizado gracias a la separación en tres niveles.	Complejo. Requiere herramientas automáticas de supervisión.
Coste	Económico. Requiere hardware básico.	Medio. Exige al menos dos servidores.	Elevado. Requiere infraestructura en la nube.
Seguridad	Baja. La base de datos está expuesta en el servidor.	Alta. La base de datos está protegida tras la aplicación.	Muy alta. Permite aplicar cifrado y permisos por capa.

Parte B: Protocolos de Desarrollo Web

1. Breve historia y evolución

Netscape creó el protocolo HTTPS en el año 1994. Su objetivo era proteger las transferencias de datos web. Para lograrlo, combinó HTTP con el cifrado SSL. Con los años, SSL mostró fallos de seguridad. Por eso, la industria desarrolló el estándar TLS.

- **SSL 2.0 y 3.0:** Primeras versiones de cifrado. Hoy están totalmente obsoletas.
- **TLS 1.2 (2008):** Fue el estándar principal durante años. Mejoró el rendimiento y la seguridad.
- **TLS 1.3 (2018):** Es la versión actual. Acelera la conexión y elimina cifrados antiguos.

2. Cómo funciona y para qué se utiliza

HTTPS es la versión cifrada del protocolo HTTP. Funciona mediante el modelo de comunicación petición/respuesta dentro de la arquitectura cliente-servidor. Su puerto por defecto es el **443** (a diferencia del puerto 80 empleado por HTTP).

Para funcionar, el servidor necesita un **certificado digital SSL/TLS** válido emitido por una Autoridad de Certificación. El proceso de comunicación (*TLS Handshake*) sigue estos pasos:

1. **Client Hello:** El navegador contacta con el servidor y envía sus opciones de cifrado.
2. **Server Hello:** El servidor responde y envía su certificado digital.
3. **Clave de sesión:** Ambos extremos crean una clave simétrica temporal. Con esta clave cifran todo el tráfico HTTP posterior (peticiones GET, POST y datos).

3. Ventajas y riesgos de seguridad

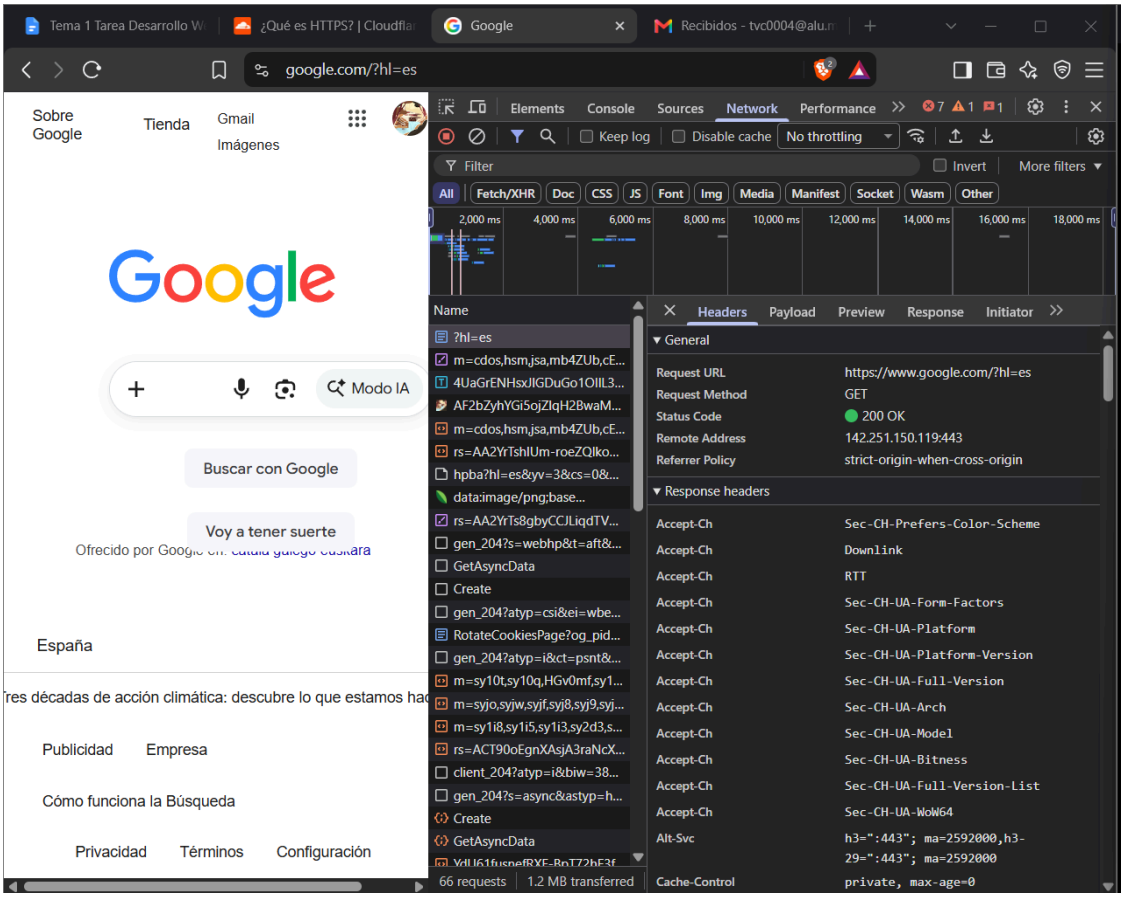
Ventajas:

- **Confidencialidad:** Los datos viajan cifrados e ilegibles para terceros.
- **Integridad:** Garantiza que la información no cambia durante el envío.
- **Autenticación:** Demuestra que el servidor es legítimo.

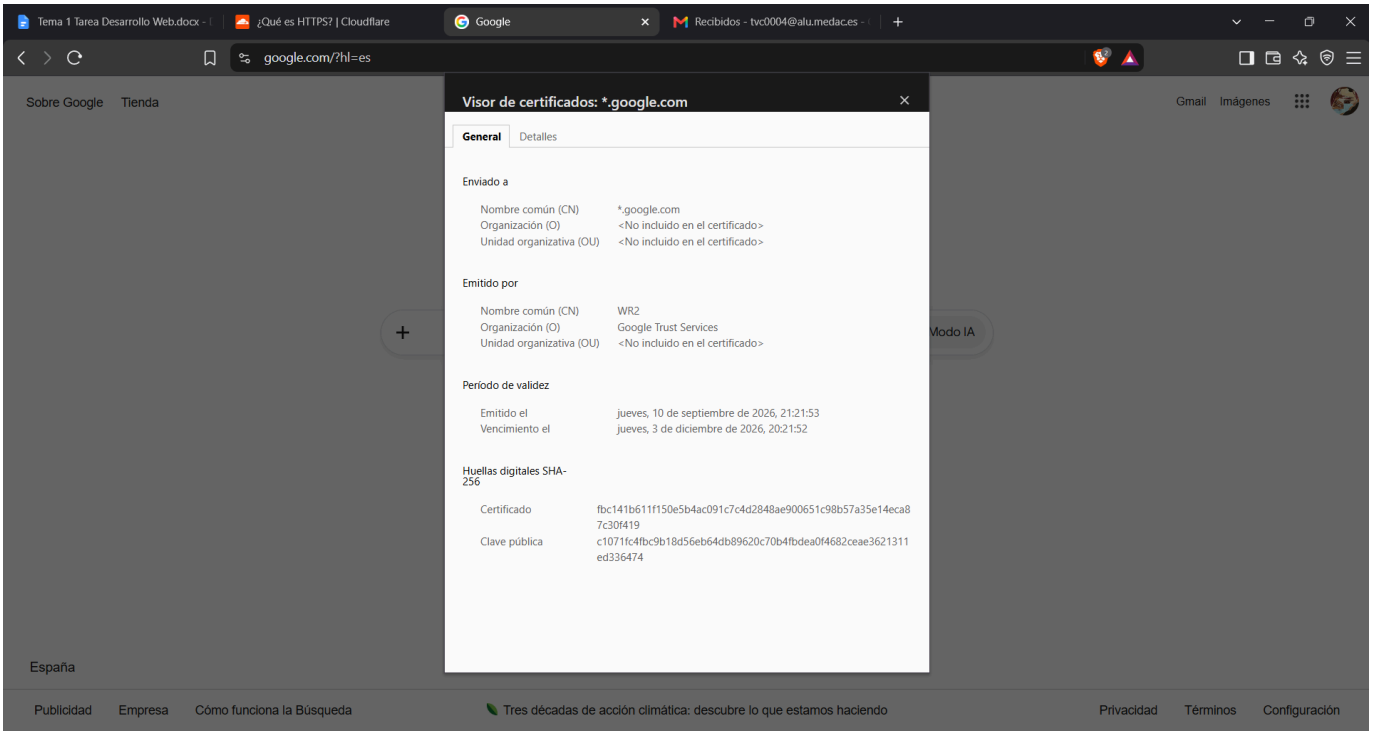
Riesgos y mitigación:

- **Ataques Man-in-the-Middle:** Un tercero intenta interceptar el tráfico en redes públicas. Se evita activando la cabecera **HSTS** en el servidor.
- **Certificados caducados:** Provocan alertas en el navegador. Se resuelve usando herramientas de renovación automática como *Certbot*.

4. Ejemplos prácticos de uso documentados



Captura de pantalla propia realizada utilizando las herramientas de desarrollador de Google Chrome (F12). En ella se puede observar una petición realizada mediante el protocolo HTTPS a través del puerto 443, la respuesta exitosa del servidor con código `200 OK` y la presencia de la cabecera de seguridad



Captura de pantalla propia realizada en el navegador mostrando las propiedades del certificado digital SSL/TLS del sitio web. En ella se puede verificar la identidad del servidor, la Autoridad de Certificación emisora de la firma digital y el periodo de validez del certificado.

5. Comparativa breve: HTTPS vs SSH

Los dos protocolos usan cifrado para proteger la información en tránsito:

- **HTTPS (Puerto 443):** Transfiere páginas y archivos web entre servidores y navegadores. Está enfocado a los usuarios finales.
- **SSH (Puerto 22):** Permite administrar servidores de forma remota mediante la terminal. Está enfocado a desarrolladores y administradores.

Conclusión: Usamos **HTTPS** para mostrar la aplicación web al público y **SSH** para gestionar el servidor de forma interna.

Uso de la Inteligencia Artificial

Herramienta utilizada: Gemini.

Uso realizado: Utilizada exclusivamente como herramienta de apoyo puntual para traducir y adaptar ciertas expresiones al español y para ayudarme algunas partes.

Bibliografía

1. **Rodero, D.** *Tema 1: El desarrollo web*. Apuntes de la asignatura Desarrollo Web en Entorno Cliente, MEDAC.
2. **Mozilla Developer Network (MDN).** *Generalidades sobre el protocolo HTTP/HTTPS*. Disponible en: https://developer.mozilla.org/es/docs/Web/HTTP/Overview
3. **AWS Documentation.** *¿Qué es una arquitectura de 3 capas?*. Disponible en: https://aws.amazon.com/es/cloudops/mobile-and-web-apps/3-tier-architecture/
4. **Cloudflare Learning Center.** *¿Cómo funciona el cifrado TLS/SSL?*. Disponible en: https://www.cloudflare.com/es-es/learning/ssl/how-does-ssl-work/